



Module 2

Deep Learning (Visvesvaraya Technological University)



Scan to open on Studocu

MODULE-2

Multilayer Perceptrons: Introduction, Batch Learning and On-Line Learning, The Back-Propagation Algorithm, XOR Problem, Heuristics for Making the Back- Propagation Algorithm Perform Better, Back Propagation and Differentiation.

2.1 Introduction

- The multilayer perceptron, which stands for a neural network with one or more hidden layers.
- In previous studies, we studied Rosenblatt's perceptron, which is basically a single-layer neural network, and this network is limited to the classification of linearly separable patterns.
- The LMS algorithm is also based on a single linear neuron with adjustable weights, which *limits the computing power of the algorithm*.
- To overcome the practical limitations of the perceptron and the LMS algorithm, we look to a neural network structure known as the multilayer perceptron.

2.1.1 The basic features of a multilayer perceptron:

- . The model of each neuron in the network includes a differentiable nonlinear activation function.
- The network contains one or more layers that are hidden from both the input and output nodes.
- The network exhibits a high degree of connectivity, the extent of which is determined by the the synaptic weights of the network.

The training of multilayer perceptron

- The training of multilayer perceptron is the back-propagation algorithm, which includes the *LMS algorithm*.
- The training proceeds in two phases:

1. **Forward phase: Input - Output (prediction)**
2. **Backward pass: Output error - Gradient - Weight update**

1. Forward phase:

- In the **forward phase**, the synaptic weights of the network are fixed
- The input signal is propagated through the network, layer by layer, until it reaches the output.
- Thus, in this phase, changes are confined to the activation potentials and outputs of the neurons in the network.
- Computes outputs layer by layer until final output.

2. Backward phase

- In the backward phase, an error signal is produced by comparing the output of the network with a desired response.
- The resulting error signal is propagated through the network, again layer by layer, but this time the propagation is performed in the backward direction.
- In this second phase, successive adjustments are made to the synaptic weights of the network.
- In this phase, the error is **propagated backward** through the network to update the weights.
- The update is done using **Gradient Descent**.
- *Compute errors at the output layer, propagate them back, and update weights using LMS rule.*
- Usage of the term “back propagation” appears to have evolved after 1985, when the term was popularized through the publication of the seminal book entitled Parallel Distributed Processing (Rumelhart and McClelland, 1986).
- The development of the back-propagation algorithm in the mid-1980s represented a landmark in neural networks in that it provided a computationally efficient method for the training of multilayer perceptrons, putting to rest the pessimism about learning in multilayer perceptrons that may have been inferred from the book by Minsky and Papert (1969).

2.1.2 Multilayer perceptron

- Multilayer perceptron with two hidden layers and an output layer.
- Fully connected.
- This means that a neuron in any layer of the network is connected to all the neurons (nodes) in the previous layer.
- Signal flow through the network progresses in a forward direction, from left to right and on a layer-by-layer basis.

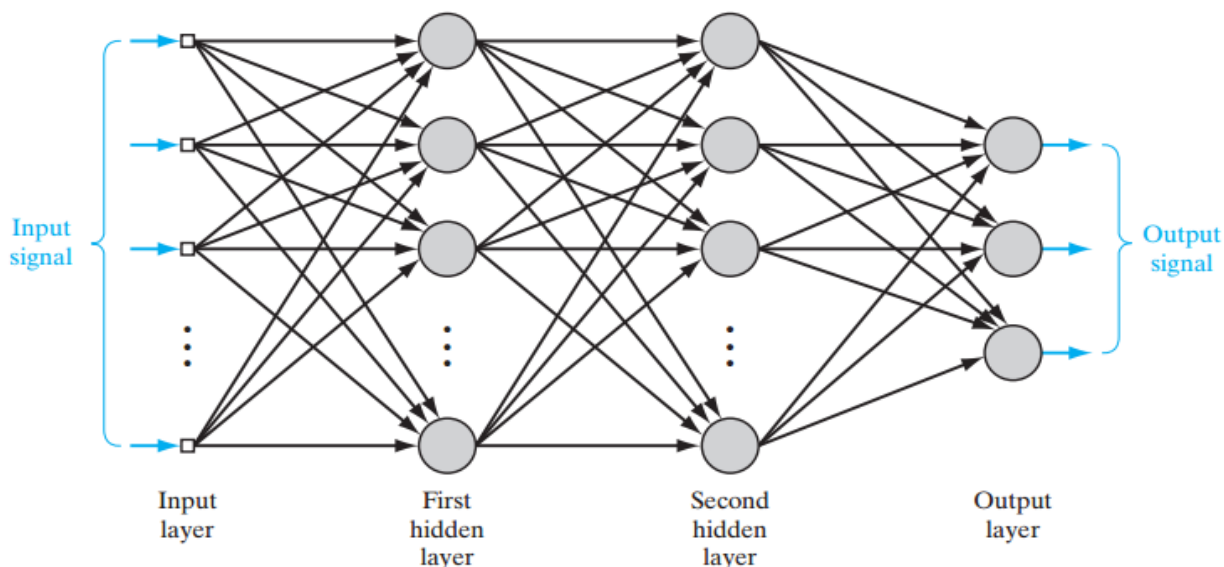
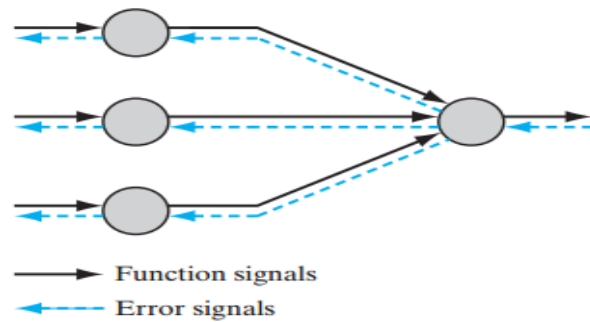


FIGURE 4.1 Architectural graph of a multilayer perceptron with two hidden layers.

Two kinds of signals are identified in this network:

1. Function Signals
2. Error Signals.



Function Signals:

- A function signal is an input signal (stimulus) that comes in at the input end of the network, propagates forward (neuron by neuron) through the network, and emerges at the output end of the network as an output signal.
- **“Function signal” for two reasons.**
 - First, it is presumed to perform a useful function at the output of the network.
 - Second, at each neuron of the network through which a function signal passes, the signal is calculated as a function of the inputs and associated weights applied to that neuron.
- The *function signal* is also referred to as the *input signal*.

Error Signals:

- An error signal originates at an output neuron of the network and propagates backward (layer by layer) through the network.
- We refer to it as an “error signal” because its computation by every neuron of the network involves an error-dependent function in one form or another.
- The output neurons constitute the output layer of the network.
- The remaining neurons constitute hidden layers of the network.
- Thus, the hidden units are not part of the output or input of the network—hence their designation as “hidden.”
- The first hidden layer is fed from the input layer made up of sensory units (source nodes)
- The resulting outputs of the first hidden layer are in turn applied to the next hidden layer and so on for the rest of the network.

Each hidden or output neuron of a multilayer perceptron is designed to perform two computations:

1. The computation of the function signal appearing at the output of each neuron, which is expressed as a continuous nonlinear function of the input signal and synaptic weights associated with that neuron
2. The computation of an *estimate of the gradient vector* (i.e., the gradients of the error surface concerning the weights connected to the inputs of a neuron), which is needed for the backward pass through the network.

2.2 Batch Learning vs. Online Learning

1. Batch Learning

Definition: The learning algorithm updates the weights **after processing the entire training dataset (one epoch)**.

- Feed all training samples through the network.
- Compute the total error (e.g., Mean Squared Error).
- Update the weights **once per epoch**.

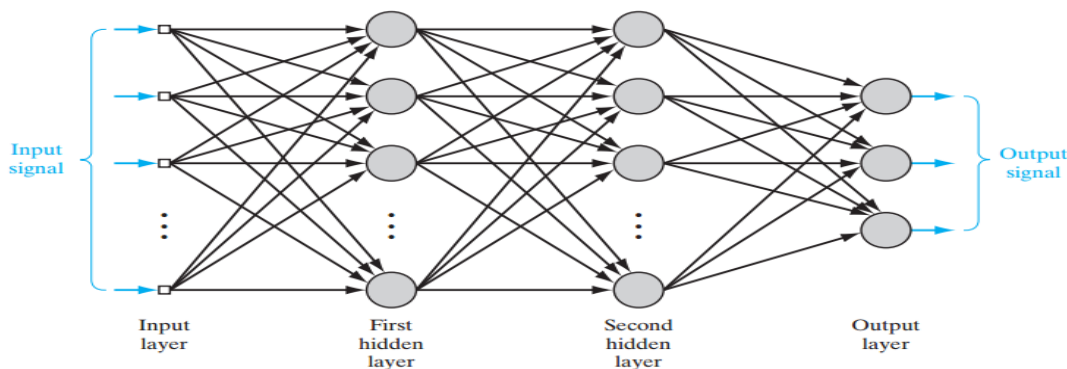
2. Online Learning (Stochastic Learning)

Definition: The weights are updated **immediately after each training sample** is presented.

- For each training example:
 - Feed forward.
 - Compute error.
 - Update weights **instantly**.

BATCH LEARNING AND ONLINE LEARNING

- Consider a multilayer perceptron with an input layer of source nodes, one or more hidden layers, and an output layer consisting of one or more neurons



Training sample

$$\mathcal{T} = \{\mathbf{x}(n), \mathbf{d}(n)\}_{n=1}^N \quad \text{.....(2.1)}$$

$$e_j(n) = d_j(n) - y_j(n) \quad \text{.....(2.2)}$$

$$\mathcal{E}_j(n) = \frac{1}{2} e_j^2(n) \quad \text{.....(2.3)}$$

Instantaneous error

$$\begin{aligned} \mathcal{E}(n) &= \sum_{j \in C} \mathcal{E}_j(n) \\ &= \frac{1}{2} \sum_{j \in C} e_j^2(n) \end{aligned} \quad \text{.....(2.4)}$$

Average Error

$$\begin{aligned} \mathcal{E}_{av}(N) &= \frac{1}{N} \sum_{n=1}^N \mathcal{E}(n) \\ &= \frac{1}{2N} \sum_{n=1}^N \sum_{j \in C} e_j^2(n) \end{aligned} \quad \text{.....(2.5)}$$

- Both **instantaneous error** $\mathcal{E}(n)$ and **average error** depend on the **weights** of the MLP.
- That's why we apply **backpropagation + gradient descent**:
 - Compute gradient of error w.r.t. weights.
 - Adjust weights to reduce error.

Batch Learning

In batch learning, the **weights of the neural network** are **not updated immediately** after each training sample.

Instead:

- The algorithm waits until **all N training samples** (one epoch) are processed.
- It calculates the **average error energy** ($\mathcal{E}_{av}$) across the whole dataset.
- Then performs one weight update step (using gradient descent).

This is why the cost function for batch learning is defined as:

$$e_{av} = \frac{1}{N} \sum_{n=1}^N \mathcal{E}(n) \quad \text{.....(2.6)}$$

where $e(n)$ is the error for the n -th training sample.

Example:

- Training a CNN for medical image classification:

- Hospital dataset of X-ray images (fixed, not changing).
- Whole dataset is used → update weights after each epoch.
- Goal: precise convergence to minimize diagnosis errors.

Online Learning (Stochastic Learning)

In online learning, the network updates its weights immediately after each training example, instead of waiting for the whole dataset (epoch).

. Cost function minimized: instantaneous error energy

$$e(n) = \frac{1}{2}(d(n) - y(n))^2 \quad \dots\dots\dots(2.7)$$

where $d(n)$ is desired output and $y(n)$ is network output for input $x(n)$.

Training Process

1. Take the **first example** $(x(1), d(1))$.
→ Forward pass → Compute error → Update weights immediately.
2. Move to **second example** $(x(2), d(2))$.
→ Repeat process (forward + error + update).
3. Continue until all **N examples** in the epoch are processed.
4. Shuffle training data and repeat for next epoch.

Example:

- **Spam email filtering:**
 - New emails arrive continuously.
 - Update model weights **immediately** after each email is classified.
 - Goal: adapt quickly to new spam techniques (nonstationary environment).

Difference between Batch Learning and On-line Learning

Aspect	Batch Learning	On-line Learning
Weight update	After all N examples (one epoch)	After each example
Gradient estimate	Accurate (average error)	Noisy (instantaneous error)
Storage	High (whole dataset needed)	Low (one sample at a time)
Convergence	Smooth, stable	Fluctuates, but can escape local minima
Parallelization	Easy (use GPUs)	Difficult
Best for	Regression, small/medium datasets	Large-scale, real-time classification
Examples	Offline training of CNNs for medical image analysis	Online spam filtering, stock market prediction

2.3 THE BACK-PROPAGATION ALGORITHM

- Backpropagation is the **learning algorithm** used to train **multilayer perceptrons (MLPs)**.
- It updates the weights of the network using **gradient descent** by propagating the error backwards from the output layer to the hidden layers.

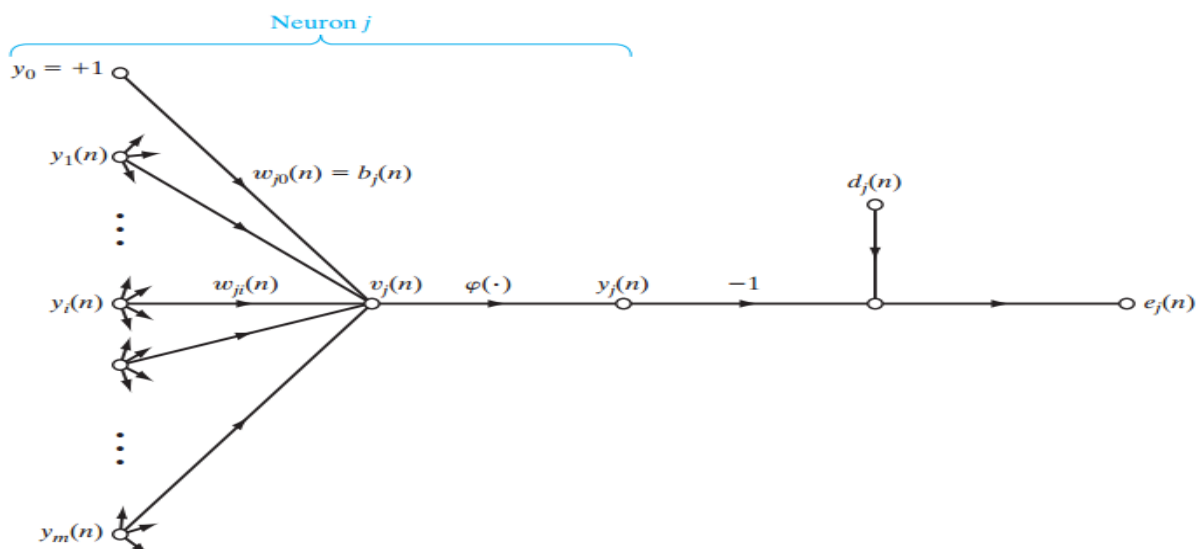


FIGURE 4.3 Signal-flow graph highlighting the details of output neuron j .

- Neuron j is being fed by a set of function signals produced by a layer of neurons to its left.
- The induced local field $v_j(n)$ produced at the input of the activation function associated with neuron j is

$$v_j(n) = \sum_{i=0}^m w_{ji}(n) y_i(n) \quad \dots\dots\dots(2.8)$$

$y_0 = +1$ (bias input)

$w_{j0}(n) = b_j(n)$ (bias weight)

- The function signal $y_j(n)$ appearing at the output of neuron j at iteration n is
The neuron output is obtained by applying the activation function $\varphi_j(\cdot)$

$$y_j(n) = \varphi_j(v_j(n)) \quad \dots\dots\dots(2.9)$$

- If neuron j is an **output neuron**, it has a desired response $d_j(n)$.
- The error signal is:

$$e_j(n) = d_j(n) - y_j(n) \quad \dots\dots\dots(2.10)$$

The **global error function** is usually:

$$E(n) = \frac{1}{2} \sum_j e_j^2(n) \quad \dots\dots\dots(2.11)$$

The weight update in backpropagation depends on the **gradient of the error $\mathcal{E}(n)$** with respect to the weight $w_{ji}(n)$:

$$\frac{\partial \mathcal{E}(n)}{\partial w_{ji}(n)} \quad \dots\dots\dots(2.12)$$

Applying the Chain Rule

We expand this derivative into multiple factors:

$$\frac{\partial \mathcal{E}(n)}{\partial w_{ji}(n)} = \frac{\partial \mathcal{E}(n)}{\partial e_j(n)} \cdot \frac{\partial e_j(n)}{\partial y_j(n)} \cdot \frac{\partial y_j(n)}{\partial v_j(n)} \cdot \frac{\partial v_j(n)}{\partial w_{ji}(n)} \quad \dots\dots\dots(2.13)$$

This shows how sensitive the error is to the weight W_j

Partially differentiating each term in Eqn(2.13)

1. $\frac{\partial \mathcal{E}(n)}{\partial e_j(n)}$

- Error function $\mathcal{E}(n)$ is usually:

$$\mathcal{E}(n) = \frac{1}{2} \sum_j e_j^2(n)$$

$$\frac{\partial \mathcal{E}(n)}{\partial e_j(n)} = e_j(n) \quad \dots\dots\dots(2.14)$$

2. $\frac{\partial e_j(n)}{\partial y_j(n)}$

- Error at output neuron:

$$e_j(n) = d_j(n) - y_j(n)$$

$$\frac{\partial e_j(n)}{\partial y_j(n)} = -1 \quad \dots\dots\dots(2.15)$$

3. $\frac{\partial y_j(n)}{\partial v_j(n)}$

- Output of neuron j :

$$y_j(n) = \varphi_j(v_j(n))$$

$$\frac{\partial y_j(n)}{\partial v_j(n)} = \varphi'_j(v_j(n)) \quad \dots\dots\dots(2.16)$$

4. $\frac{\partial v_j(n)}{\partial w_{ji}(n)}$

- Local field:

$$v_j(n) = \sum_i w_{ji}(n) y_i(n)$$

$$\frac{\partial v_j(n)}{\partial w_{ji}(n)} = y_i(n) \quad \dots\dots\dots(2.17)$$

◆ Combining All Terms

Multiply them together:

$$\begin{aligned} \frac{\partial \mathcal{E}(n)}{\partial w_{ji}(n)} &= e_j(n) \cdot (-1) \cdot \varphi'_j(v_j(n)) \cdot y_i(n) \\ &= -e_j(n) \varphi'_j(v_j(n)) y_i(n) \end{aligned}$$

Weight Update Rule (Delta Rule)

The correction to the weight:

$$\Delta w_{ji}(n) = -\eta \frac{\partial \mathcal{E}(n)}{\partial w_{ji}(n)} \quad \frac{\partial e(n)}{\partial w_{ji}(n)} = \frac{\partial e_j(n)}{\partial y_j(n)} \cdot \frac{\partial y_j(n)}{\partial v_j(n)} \cdot \frac{\partial v_j(n)}{\partial w_{ji}(n)}$$

$$\Delta w_{ji}(n) = \eta e_j(n) \varphi'_j(v_j(n)) y_i(n) \dots\dots\dots(2.18)$$

But **local gradient**

$$\begin{aligned} \delta_j(n) &= \frac{\partial \mathcal{E}(n)}{\partial v_j(n)} \\ &= \frac{\partial \mathcal{E}(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} \\ &= e_j(n) \varphi'_j(v_j(n)) \end{aligned}$$

Therefore, weight update is,

$$\Delta w_{ji}(n) = \eta \delta_j(n) y_i(n) \dots\dots\dots(2.19)$$

weight change = learning rate × local gradient × input.

Two cases of Local Gradient

Case 1: Neuron j is an output neuron

- Desired response $d_j(n)$ is available.
- Error signal is:

$$e_j(n) = d_j(n) - y_j(n)$$

- Local gradient is:

$$\delta_j(n) = e_j(n) \varphi'_j(v_j(n))$$

- Weight update:

$$\Delta w_{ji}(n) = \eta \delta_j(n) y_i(n)$$

This is straightforward because the error signal is **directly known**.

Case 2: Neuron j is a hidden neuron.

- Hidden neurons don't have direct error signals..
- Their local gradient is computed using gradients of neurons in the next layer.

The local gradient is:

$$\delta_j(n) = \varphi'_j(v_j(n)) \sum_k \delta_k(n) w_{kj}(n)$$

- $\varphi'_j(v_j(n))$: derivative of the hidden neuron's activation function
- $\delta_k(n)$: local gradients of neurons in the next layer
- $w_{kj}(n)$: weights connecting hidden neuron j to those neurons k

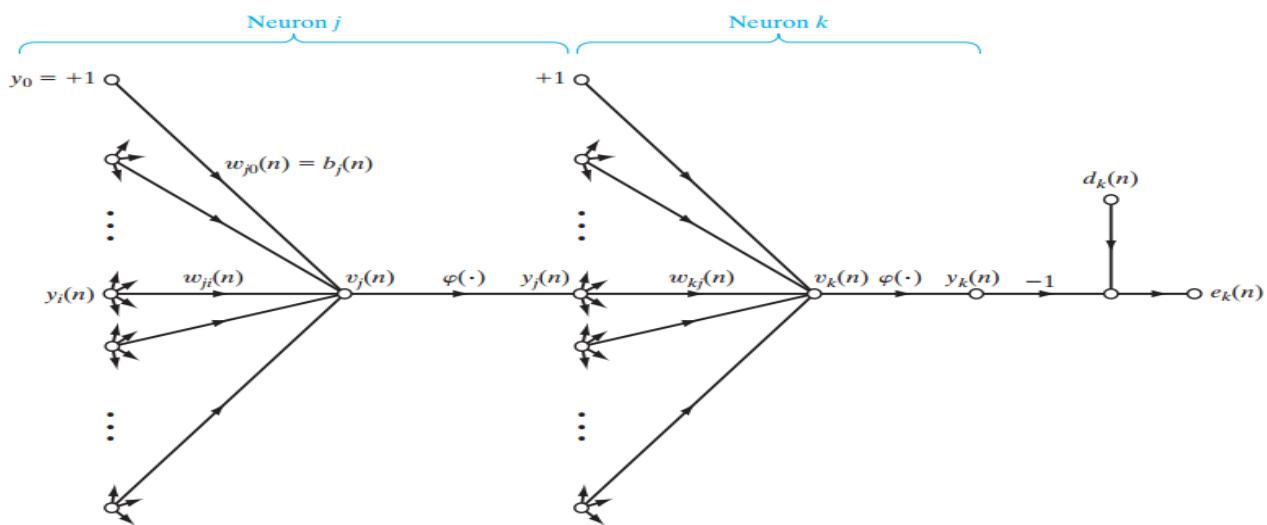


FIGURE 4.4 Signal-flow graph highlighting the details of output neuron k connected to hidden neuron j .

$$\begin{aligned} \delta_j(n) &= - \frac{\partial \mathcal{E}(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} \\ &= - \frac{\partial \mathcal{E}(n)}{\partial y_j(n)} \varphi'_j(v_j(n)), \quad \text{neuron } j \text{ is hidden} \end{aligned}$$

$$\mathcal{E}(n) = \frac{1}{2} \sum_{k \in C} e_k^2(n), \quad \text{neuron } k \text{ is an output node}$$

$$\frac{\partial \mathcal{E}(n)}{\partial y_j(n)} = \sum_k e_k \frac{\partial e_k(n)}{\partial y_j(n)} \quad \text{use the chain rule for the partial derivative } \frac{\partial \mathcal{E}(n)}{\partial y_j(n)}$$

Next we use the chain rule for the partial derivative $\partial e_k(n) / \partial y_j(n)$

$$\frac{\partial e_k(n)}{\partial y_j(n)} = \frac{\partial e_k}{\partial y_k} \cdot \frac{\partial y_k}{\partial v_k} \cdot \frac{\partial v_k}{\partial y_j}$$

$$\frac{\partial e_k(n)}{\partial y_j(n)} = -\varphi'_k(v_k(n))w_{kj}(n)$$

$$\begin{aligned} \frac{\partial \mathcal{E}(n)}{\partial y_j(n)} &= \sum_k e_k(n) \frac{\partial e_k(n)}{\partial y_j(n)} \\ &= -\sum_k e_k(n) \varphi'_k(v_k(n)) w_{kj}(n) \\ &= -\sum_k \delta_k(n) w_{kj}(n) \end{aligned}$$

$$\delta_j(n) = \varphi'_j(v_j(n)) \sum_k \delta_k(n) w_{kj}(n), \quad \text{neuron } j \text{ is hidden}$$

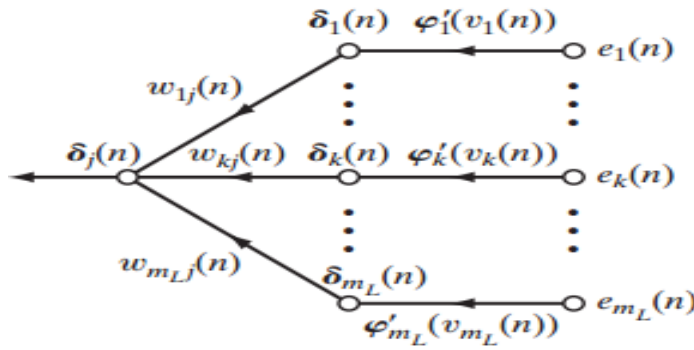


FIGURE 4.5 Signal-flow graph of a part of the adjoint system pertaining to backpropagation of error signals.

$$\begin{pmatrix} \text{Weight} \\ \text{correction} \\ \Delta w_{ji}(n) \end{pmatrix} = \begin{pmatrix} \text{learning-} \\ \text{rate parameter} \\ \eta \end{pmatrix} \times \begin{pmatrix} \text{local} \\ \text{gradient} \\ \delta_j(n) \end{pmatrix} \times \begin{pmatrix} \text{input signal} \\ \text{of neuron } j, \\ y_i(n) \end{pmatrix}$$

The Two Passes of Computation

1. Forward Pass

We start with the input vector $x(n)$ and compute outputs layer by layer.

- **Net input to each neuron:**

$$v_j(n) = \sum_{i=0}^m w_{ji}(n) y_i(n)$$

Here:

- $w_{ji}(n)$: weight from neuron i to neuron j
- $y_i(n)$: output from the previous layer neuron i
- **Output of neuron j using activation function φ :**

$$y_j(n) = \varphi(v_j(n))$$

- **Error at the output layer:**

$$e_j(n) = d_j(n) - y_j(n)$$

where $d_j(n)$ is the desired output.

2. Backward Pass

Now we propagate the error back to update the weights.

1. At the output layer:

$$\delta_j(n) = e_j(n) \varphi'_j(v_j(n))$$

This is the *local gradient* for output neurons.

2. For hidden layers:

$$\delta_j(n) = \varphi'_j(v_j(n)) \sum_k \delta_k(n) w_{kj}(n)$$

This recursively propagates error from the next layer k back to hidden layer neuron j .

3. Weight Update Rule:

$$\Delta w_{ji}(n) = \eta \delta_j(n) y_i(n)$$

where η is the learning rate.

Activation Function

An **activation function** is a mathematical function applied to the weighted sum of a neuron's inputs in a neural network to determine its **output signal**. It introduces **non-linearity** into the model so the network can learn complex patterns rather than just simple linear relationships.

Each neuron in a multilayer perceptron (MLP) requires the **derivative of its activation function** $\phi(\cdot)$ to compute weight updates.

1. Logistic Function. This form of sigmoidal nonlinearity, in its general form, is defined by

$$\varphi_j(v_j(n)) = \frac{1}{1 + \exp(-av_j(n))}, \quad a > 0 \quad (4.32)$$

$v_j(n)$ – induced local field of neuron j .

a – adjustable positive parameter.

Output amplitude lies in the range $0 \leq y_j \leq 1$.

Differentiate w.r.t. $v_j(n)$:

$$\phi'_j(v_j(n)) = \frac{a \exp[-av_j(n)]}{[1 + \exp(-av_j(n))]^2}.$$

Using $y_j(n) = \phi_j(v_j(n))$:

$$\phi'_j(v_j(n)) = a y_j(n) [1 - y_j(n)].$$

Local gradient – output neuron j

- With $y_j(n) = o_j(n)$ (output signal) and error $e_j(n) = d_j(n) - o_j(n)$:

$$\delta_j(n) = e_j(n) \phi'_j(v_j(n)) = a [d_j(n) - o_j(n)] o_j(n) [1 - o_j(n)].$$

Local gradient – hidden neuron j

- Weighted sum of downstream errors:

$$\delta_j(n) = \phi'_j(v_j(n)) \sum_k \delta_k(n) w_{kj}(n) = a y_j(n) [1 - y_j(n)] \sum_k \delta_k(n) w_{kj}(n)$$

2. Hyperbolic tangent function. Another commonly used form of sigmoidal non-linearity is the hyperbolic tangent function, which, in its most general form, is defined by

$$\varphi_j(v_j(n)) = a \tanh(bv_j(n)) \quad (4.37)$$

where a and b are positive constants. In reality, the hyperbolic tangent function is just the logistic function rescaled and biased. Its derivative with respect to $v_j(n)$ is given by

$$\begin{aligned} \varphi'_j(v_j(n)) &= ab \operatorname{sech}^2(bv_j(n)) \\ &= ab(1 - \tanh^2(bv_j(n))) \\ &= \frac{b}{a} [a - y_j(n)][a + y_j(n)] \end{aligned} \quad (4.38)$$

For a neuron j located in the output layer, the local gradient is

$$\begin{aligned} \delta_j(n) &= e_j(n) \varphi'_j(v_j(n)) \\ &= \frac{b}{a} [d_j(n) - o_j(n)][a - o_j(n)][a + o_j(n)] \end{aligned} \quad (4.39)$$

For a neuron j in a hidden layer, we have

$$\begin{aligned} \delta_j(n) &= \varphi'_j(v_j(n)) \sum_k \delta_k(n) w_{kj}(n) \\ &= \frac{b}{a} [a - y_j(n)][a + y_j(n)] \sum_k \delta_k(n) w_{kj}(n), \quad \text{neuron } j \text{ is hidden} \end{aligned} \quad (4.40)$$

By using Eqs. (4.35) and (4.36) for the logistic function and Eqs. (4.39) and (4.40) for the hyperbolic tangent function, we may calculate the local gradient δ_j without requiring explicit knowledge of the activation function.

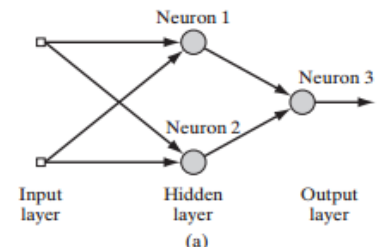
XOR PROBLEM

Single-layer perceptron limitation

- Rosenblatt's perceptron has **no hidden layer**.
- Its decision boundary is a **straight line** in 2-D input space (or a hyperplane in higher dimensions).
- Therefore, it can only classify **linearly separable** patterns.

XOR problem

- Inputs: four corners of the unit square — (0,0), (0,1), (1,0), (1,1).
- Desired outputs:
 - Class 0 $\rightarrow$ (0,0) and (1,1).
 - Class 1 $\rightarrow$ (0,1) and (1,0).

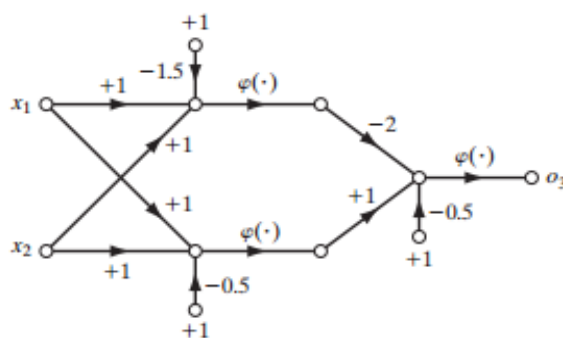


- These points **cannot** be separated by a single straight line $\rightarrow$ **not linearly separable**.
- A single neuron produces output 1 on one side of its linear boundary and 0 on the other. No matter how the line is placed, it cannot group (0,0) with (1,1) separately from (0,1) and (1,0).
- Introduce **one hidden layer with two neurons** (Touretzky & Pomerleau, 1989). Each hidden neuron implements its own linear boundary. The combination of their outputs allows the network to create a **non-linear (piecewise linear)** decision region that solves XOR.

Hidden neurons are **McCulloch–Pitts units** using a **threshold activation**. Logical bits: 0 $\rightarrow$ level 0, 1 $\rightarrow$ level +1.

One hidden neuron creates a boundary of **slope -1**; the second neuron forms a complementary boundary. Proper weights and biases (e.g., $b_2 = -1/2$) position these boundaries so their outputs can be combined in the output neuron to implement the XOR function.

•



The top neuron, labeled as “Neuron 1” in the hidden layer, is characterized as

$$w_{11} = w_{12} = +1$$
$$b_1 = -\frac{3}{2}$$

The slope of the decision boundary constructed by this hidden neuron is equal to -1 and positioned as in fig(a). The bottom neuron, labeled as “Neuron 2” in the hidden layer, is characterized as

$$w_{21} = w_{22} = +1$$

$$b_2 = -\frac{1}{2}$$

The output neuron, labeled as “Neuron 3” in Fig. 4.8a, is characterized as

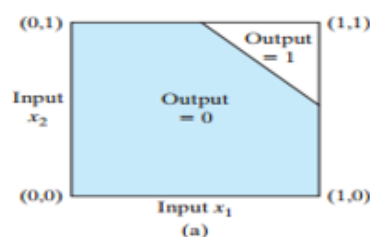
$$w_{31} = -2$$

$$w_{32} = +1$$

$$b_3 = -\frac{1}{2}$$

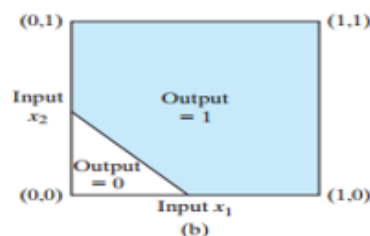
The function of the output neuron is to construct a linear combination of the decision boundaries formed by the two hidden neurons. The result of this computation is shown in Fig. 4.9c. The bottom hidden neuron has an excitatory (positive) connection to the output neuron, whereas the top hidden neuron has an inhibitory (negative) connection to the output neuron. When both hidden neurons are off, which occurs when the input pattern is (0,0), the output neuron remains off. When both hidden neurons are on, which occurs when the input pattern is (1,1), the output neuron is switched off again because the inhibitory effect of the larger negative weight connected to the top hidden neuron overpowers the excitatory effect of the positive weight connected to the bottom hidden neuron. When the top hidden neuron is off and the bottom hidden neuron is on, which occurs when the input pattern is (0,1) or (1,0), the output neuron is switched on because of the excitatory effect of the positive weight connected to the bottom hidden neuron. Thus, the network of Fig. 4.8a does indeed solve the XOR problem.

FIGURE 4.9 (a) Decision boundary constructed by hidden neuron 1 of the network in Fig. 4.8. (b) Decision boundary constructed by hidden neuron 2 of the network. (c) Decision boundaries constructed by the complete network.



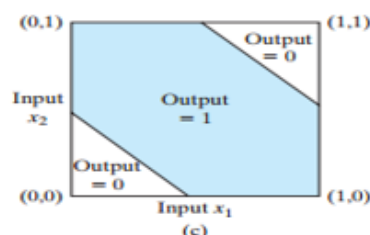
(a) Hidden neuron 1 boundary:

Defines a line with slope -1 . It separates the top-right triangle (output=1) from the bottom-left region (output=0).



(b) Hidden neuron 2 boundary:

Defines another line with slope -1 but at a different intercept. It separates a diagonal region into “1” and “0.”



(c) When you **combine** the outputs of both hidden neurons (via the output neuron), the two linear cuts divide the square into **four regions**. Now the network can **label two opposite corners** as class 0 and the other two opposite corners as class 1.

-]
- Hidden Neuron 2 = Second line (roughly selects the other diagonal).

- Output Neuron = Combines these two decisions (one excitatory weight, one inhibitory weight) to realize XOR.

Thus, we need **two lines** because XOR is **not linearly separable**. Each hidden neuron contributes one line (one linear cut), and their combination allows the network to create a **nonlinear decision boundary** that solves XOR.

Problem:

- Figure P4.1 shows a neural network involving a single hidden neuron for solving the XOR problem; this network may be viewed as an alternative to that considered in Section 4.5. Show that the network of Fig. P4.1 solves the XOR problem by constructing

- (a) decision regions and
- (b) a truth table for the network.

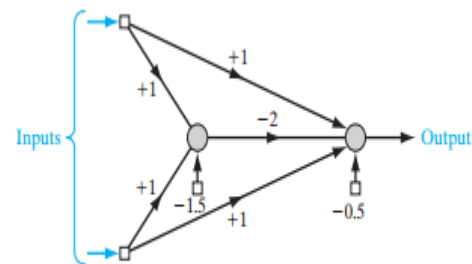


FIGURE P4.1

Heuristics for making the

1. Stochastic versus batch update
2. Maximizing information content- the largest possible for the task
3. Activation function
4. Target values.
5. Normalizing the inputs
6. Initialization.
7. Learning from hints
8. Learning rates

1. Stochastic versus batch update:

Stochastic (sequential) mode: Updates network weights **after every training pattern**.

Batch mode: Accumulates gradients over the **entire training set** and updates once per epoch.

2. Maximizing information content.

Every training example presented to the back-propagation algorithm should be chosen on the basis that its information content is the largest possible for the task at hand (LeCun, 1993).

Two ways of realizing this choice are as follows:

- Use an example that results in the largest training error.
- Use an example that is radically different from all those previously used.

These two heuristics are motivated by a desire to search more of the weight space. In pattern-classification tasks using sequential back-propagation learning, a simple and commonly used technique is to randomize (i.e., shuffle) the order in which the examples are presented to the multilayer perceptron from one epoch to the next. Ideally, the randomization ensure that successive examples in an epoch presented to the network rarely belong to the same class.

3. Activation function.

Insofar as the speed of learning is concerned, the preferred choice is to use a sigmoid activation function that is an *odd function of its argument*, as shown by

$$\varphi(-v) = -\varphi(v)$$

This condition is satisfied by the hyperbolic function

$$\varphi(v) = a \tanh(bv)$$

as shown in Fig. 4.10, but not the logistic function. Suitable values for the constraints a and b in the formula for $\varphi(v)$ are as follows (LeCun, 1993):

$$a = 1.7159$$

$$b = \frac{2}{3}$$

The hyperbolic tangent function $\varphi(v)$ of Fig. 4.10 has the following useful properties:

- $\varphi(1) = 1$ and $\varphi(-1) = -1$.
- At the origin, the slope (i.e., effective gain) of the activation function is close to unity, as shown by

$$\begin{aligned}\varphi'(0) &= ab \\ &= 1.7159 \left(\frac{2}{3} \right) \\ &= 1.1424\end{aligned}$$

- The second derivative of $\varphi(v)$ attains its maximum value at $v = 1$.

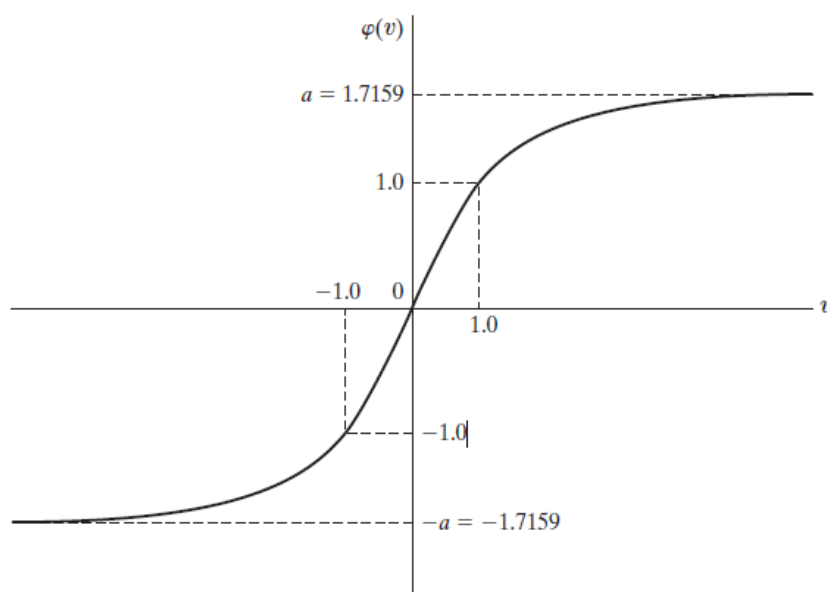


FIGURE 4.10 Graph of the hyperbolic tangent function $\varphi(v) = a \tanh(bv)$ for $a = 1.7159$ and $b = 2/3$. The recommended target values are $+1$ and -1 .

4. **Target values**. It is important that the target values (desired response) be chosen within the range of the sigmoid activation function. More specifically, the desired response d_j for neuron j in the output layer of the multilayer perceptron should be *offset* by some amount ϵ away from the limiting value of the sigmoid activation function, depending on whether the limiting value is positive or negative. Otherwise, the backpropagation algorithm tends to drive the free parameters of the network to infinity and thereby slow down the learning process by driving the hidden neurons into saturation. To be specific, consider the hyperbolic tangent function of Fig. 4.10. For the limiting value a , we set

$$d_j = a - \epsilon$$

and for the limiting value of $-a$, we set

$$d_j = -a + \epsilon$$

5. **Normalizing the inputs**. Each input variable should be *preprocessed* so that its mean value, averaged over the entire training sample, is close to zero, or else it will be small compared to its standard deviation (LeCun, 1993). To appreciate the practical significance of this rule, consider the extreme case where the input variables are consistently positive. In this situation, the synaptic weights of a neuron in the first hidden layer can only increase together or decrease together. Accordingly, if the weight vector of that neuron is to change direction, it can do so only by zigzagging its way through the error

surface, which is typically slow and should therefore be avoided.

In order to accelerate the back-propagation learning process, the normalization of the inputs should also include two other measures (LeCun, 1993):

- The input variables contained in the training set should be *uncorrelated*; this can be done by using principal-components analysis.
- The decorrelated input variables should be scaled so that their *covariances are approximately equal*, thereby ensuring that the different synaptic weights in the network learn at approximately the same speed.

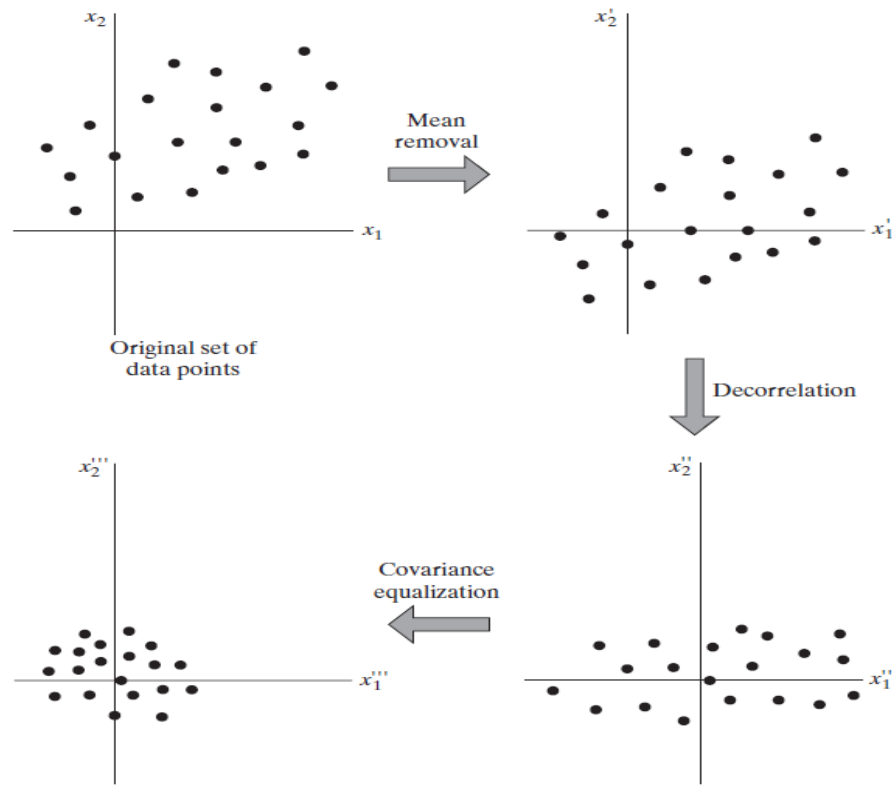


FIGURE 4.11 Illustrating the operation of mean removal, decorrelation, and covariance equalization for a two-dimensional input space.

Figure 4.11 illustrates the results of three normalization steps: mean removal, decorrelation, and covariance equalization, applied in that order.

6. Initialization. A good choice for the initial values of the synaptic weights and thresholds of the network can be of tremendous help in a successful network design.

When the synaptic weights are assigned large initial values, it is highly likely that the neurons in the network will be driven into saturation. If this happens, the local gradients in the backpropagation algorithm assume small values, which in turn will cause the learning process to slow down. However, if the synaptic weights are assigned small initial values, the back-propagation algorithm may operate on a very flat area around the origin of the error surface; this is particularly true in the case of sigmoid

functions such as the hyperbolic tangent function. Unfortunately, the origin is a *saddle point*, which refers to a stationary point where the curvature of the error surface across the saddle is negative and the curvature along the saddle is positive. For these reasons, the use of both large and small values for initializing the synaptic weights should be avoided. The proper choice of initialization lies somewhere between these two extreme cases.

To be specific, consider a multilayer perceptron using the hyperbolic tangent function for its activation functions. Let the bias applied to each neuron in the network be set to zero.

The induced local field of neuron j as

$$v_j = \sum_{i=1}^m w_{ji} y_i$$

The mean and variance of the induced local field v_j as

$$\mu_v = \mathbb{E}[v_j] = \mathbb{E}\left[\sum_{i=1}^m w_{ji}y_i\right] = \sum_{i=1}^m \mathbb{E}[w_{ji}]\mathbb{E}[y_i] = 0$$

$$\begin{aligned}\sigma_v^2 &= \mathbb{E}[(v_j - \mu_v)^2] = \mathbb{E}[v_j^2] \\ &= \mathbb{E}\left[\sum_{i=1}^m \sum_{k=1}^m w_{ji}w_{jk}y_iy_k\right] \\ &= \sum_{i=1}^m \sum_{k=1}^m \mathbb{E}[w_{ji}w_{jk}]\mathbb{E}[y_iy_k] \\ &= \sum_{i=1}^m \mathbb{E}[w_{ji}^2] \\ &= m\sigma_w^2\end{aligned}$$

7. Learning from hints.

Learning from a sample of training examples deals with an unknown input–output mapping function $f(\cdot)$. In effect, the learning process exploits the information contained in the examples about the function $f(\cdot)$ to *infer* an approximate implementation of it. The process of learning from examples may be generalized to *include learning from hints*, which is achieved by allowing prior information that we may have about the function $f(\cdot)$ to be included in the learning process. Such information may include invariance properties, symmetries, or any other knowledge about the function $f(\cdot)$ that may be used to accelerate the search for its approximate realization.

8. Learning rates.

All neurons in the multilayer perceptron should ideally learn at the same rate. The last layers usually have larger local gradients than the layers at the front end of the network. Hence, the learning-rate parameter η should be assigned a smaller value in the last layers than in the front layers of the multilayer perceptron. smaller value in the last layers than in the front layers of the multilayer perceptron.

Neurons with many inputs should have a smaller learning-rate parameter than neurons with few inputs so as to maintain a similar learning time for all neurons in the network. In LeCun (1993), it is suggested that for a given neuron, the learning rate should be inversely proportional to the square root of synaptic connections made to that neuron.

4.7 BACK PROPAGATION AND DIFFERENTIATION

- Back propagation is a specific technique for implementing gradient descent in weight space for a multilayer perceptron.
- The basic idea is to efficiently compute partial derivatives of an approximating function $F(\mathbf{w}, \mathbf{x})$ realized by the network with respect to all the elements of the adjustable weight vector $\mathbf{w}$ for a given value of input vector $\mathbf{x}$.

- Back-propagation is used because it gives an exact analytic gradient of the network output (or loss) with respect to all weights using **one forward pass + one backward pass**, exploiting the chain rule and reuse of intermediate results — making gradient-based learning feasible for multilayer networks with many parameters.
- **Back-propagation is the efficient way to compute every partial derivative**

$$\frac{\partial F(w, x)}{\partial w}$$

- The gradient of the network output or of the loss for a given **input** x .
- These partials are exactly the information gradient-descent (or second-order) optimizers need to change the **weights** w so the network output moves toward the desired target.

Layers $l=0..3$

net inputs $A_j^{(l)}$, activation $\varphi(\cdot)$, weights $w_{ji}^{(l)}$.

Network output $F(w, x)$

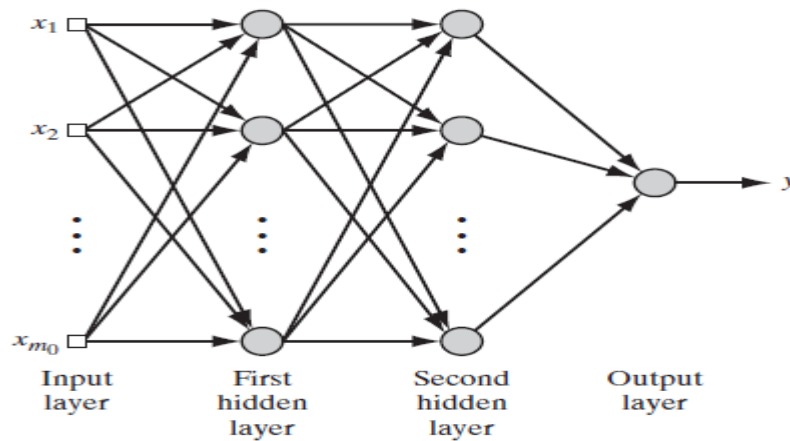


Figure: Multilayer perceptron with two hidden layers and one output neuron.

neuron. For $l = 3$, corresponding to the output layer in Fig. 4.14, we have $j = 1$. We wish to evaluate the derivatives of the function $F(\mathbf{w}, \mathbf{x})$ with respect to all the elements of the weight vector $\mathbf{w}$ for a specified input vector $\mathbf{x} = [x_1, x_2, \dots, x_{m_0}]^T$. We have included the weight vector $\mathbf{w}$ as an argument of the function F in order to focus attention on it. For example, for $l = 2$ (i.e., a single hidden layer and a linear output layer), we have

$$F(\mathbf{w}, \mathbf{x}) = \sum_{j=0}^{m_1} w_{oj} \varphi \left(\sum_{i=0}^{m_0} w_{ji} x_i \right) \quad (4.49)$$

where $\mathbf{w}$ is the ordered weight vector and $\mathbf{x}$ is the input vector.

The multilayer perceptron of Fig. 4.14 is parameterized by an *architecture* $\mathcal{A}$ (representing a discrete parameter) and a *weight vector* $\mathbf{w}$ (made up of continuous elements). Let $\mathcal{A}_j^{(l)}$ denote that part of the architecture extending from the input layer ($l = 0$) to node j in layer $l = 1, 2, 3$. Accordingly, we may write

$$F(\mathbf{w}, \mathbf{x}) = \varphi(\mathcal{A}_1^{(3)}) \quad (4.50)$$

where φ is the activation function. However, $\mathcal{A}_1^{(3)}$ is to be interpreted merely as an architectural symbol rather than a variable. Thus, adapting Eqs. (4.2), (4.4), (4.13), and (4.25) for use in this new situation, we obtain the formulas

$$\frac{\partial F(\mathbf{w}, \mathbf{x})}{\partial w_{1k}^{(3)}} = \varphi'(\mathcal{A}_1^{(3)}) \varphi(\mathcal{A}_k^{(2)}) \quad (4.51)$$

$$\frac{\partial F(\mathbf{w}, \mathbf{x})}{\partial w_{kj}^{(2)}} = \varphi'(\mathcal{A}_1^{(3)}) \varphi'(\mathcal{A}_k^{(2)}) \varphi(\mathcal{A}_j^{(1)}) w_{1k}^{(3)} \quad (4.52)$$

$$\frac{\partial F(\mathbf{w}, \mathbf{x})}{\partial w_{ji}^{(1)}} = \varphi'(\mathcal{A}_1^{(3)}) \varphi'(\mathcal{A}_j^{(1)}) x_i \left[\sum_k w_{1k}^{(3)} \varphi'(\mathcal{A}_k^{(2)}) w_{kj}^{(2)} \right] \quad (4.53)$$

where φ' is the partial derivative of the nonlinearity φ with respect to its argument and x_i is the i th element of the input vector $\mathbf{x}$. In a similar way, we may derive the equations for the partial derivatives of a general network with more hidden layers and more neurons in the output layer.

Equations (4.51) through (4.53) provide the basis for calculating the sensitivity of the network function $F(\mathbf{w}, \mathbf{x})$ with respect to variations in the elements of the weight vector $\mathbf{w}$. Let ω denote an element of the weight vector $\mathbf{w}$. The *sensitivity* of $F(\mathbf{w}, \mathbf{x})$ with respect to ω , is formally defined by

$$S_\omega^F = \frac{\partial F / F}{\partial \omega / \omega}$$

It is for this reason that we refer to the lower part of the signal-flow graph in Fig. 4.7 as a “sensitivity graph.”